

**UNIVERSIDAD MESOAMERICANA SEDE
QUETZALTENANGO
FACULTAD DE INGENIERÍA**

**INGENIERÍA EN SISTEMAS, INFORMÁTICA Y CIENCIAS
DE LA COMPUTACIÓN**



**NETGUARDIAN OS: DISTRIBUCIÓN LINUX
PERSONALIZADA PARA EL MONITOREO Y
DIAGNÓSTICO DE REDES**

ESTUDIANTES:

Yoshua Emanuel González Fuentes (202408012)
Geovanny Rafael Orozco Rodas (202408012)

Guatemala, Mayo de 2026

Índice

1. Introducción	3
2. Objetivos	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
3. Marco Teórico	4
3.1. Arch Linux	4
3.2. ArchISO	4
3.3. VirtualBox	4
3.4. Systemd	4
3.5. Openbox y Chromium	5
3.6. Herramientas Integradas	5
4. Desarrollo del Proyecto	5
4.1. Creación de la Máquina Virtual de Desarrollo	5
4.2. Instalación del Sistema Base de Arch Linux	7
4.3. Configuración de Usuario y Sincronización Inicial	8
4.4. Instalación de Utilidades del Sistema y Entorno ArchISO	9
4.5. Estructuración del Perfil Live y Selección de Paquetes	10
4.6. Automatización con Systemd y Scripts de Inicialización	11
4.7. Orquestación del Monitor Hub con Docker	13
4.8. Compilación y Generación de NetGuardian OS	14
5. Pruebas Realizadas y Diagnóstico	15
5.1. Validación de Interfaces de Red y Contenedores	15
5.2. Captura Atómica y Análisis en Vivo	15
5.3. Interfaz de Usuario Final (Dashboard)	17
6. Problemas Encontrados y Soluciones	18
6.1. Problemas con Mirrors durante la Instalación	18

6.2. Arranque desde la ISO de Instalación	18
6.3. Problemas con la Carpeta Compartida	19
6.4. Uso Inicial de Netdata	19
6.5. Problemas con Docker en Entorno Live	19
7. Resultados Obtenidos	19
8. Conclusiones	20
9. Recomendaciones	20
10. Bibliografía	21

1. Introducción

En la actualidad, la administración y monitoreo de redes constituye una actividad fundamental dentro de la infraestructura tecnológica de cualquier organización. La capacidad de identificar fallos, monitorear el rendimiento de los equipos y analizar el tráfico de red permite mejorar la disponibilidad de los servicios y optimizar los recursos informáticos.

El presente proyecto consiste en el diseño e implementación de **NetGuardian OS**, una distribución Linux personalizada basada en Arch Linux, orientada al monitoreo y diagnóstico de redes. Para su desarrollo se utilizó la herramienta ArchISO, permitiendo generar una imagen ISO personalizada que integra diversas herramientas especializadas para la administración y análisis de sistemas y redes.

Durante el desarrollo se incorporaron utilidades como Nmap, Tcpdump, Htop, Iftop, Docker, Traceroute y otras herramientas de diagnóstico. Además, se implementó una interfaz gráfica personalizada que facilita el acceso a las funcionalidades principales del sistema.

El proyecto permitió aplicar conocimientos relacionados con sistemas operativos, administración de redes, virtualización, automatización de servicios mediante systemd y personalización de distribuciones Linux, proporcionando una solución práctica para el análisis y monitoreo de entornos de red.

2. Objetivos

2.1. Objetivo General

Diseñar e implementar una distribución Linux personalizada denominada **NetGuardian OS** para el monitoreo y diagnóstico de redes mediante la integración de herramientas especializadas y mecanismos de automatización.

2.2. Objetivos Específicos

- Personalizar una distribución Linux basada en Arch Linux utilizando ArchISO.
- Integrar herramientas de monitoreo y diagnóstico de red dentro de la distribución.

- Implementar mecanismos automáticos de arranque y ejecución mediante systemd.
- Desarrollar una interfaz web sencilla para facilitar el acceso a las herramientas disponibles.
- Validar el funcionamiento de la distribución mediante pruebas en máquinas virtuales.

3. Marco Teórico

3.1. Arch Linux

Arch Linux es una distribución GNU/Linux caracterizada por su simplicidad, flexibilidad y filosofía de diseño minimalista. Proporciona al usuario un alto grado de control sobre la configuración del sistema, permitiendo instalar únicamente los componentes necesarios para un propósito específico.

3.2. ArchISO

ArchISO es una herramienta oficial de Arch Linux que permite generar imágenes ISO personalizadas. Esta utilidad facilita la creación de distribuciones adaptadas a necesidades específicas mediante la incorporación de paquetes, configuraciones, scripts y servicios personalizados.

3.3. VirtualBox

VirtualBox es una plataforma de virtualización que permite ejecutar sistemas operativos invitados sobre un sistema anfitrión. En este proyecto fue utilizada para instalar Arch Linux, construir la ISO personalizada y validar el funcionamiento de NetGuardian OS en un entorno controlado.

3.4. Systemd

Systemd es un sistema de inicialización y administración de servicios utilizado por numerosas distribuciones Linux. Permite automatizar procesos de arranque, ejecución de scripts y

administración de servicios del sistema.

3.5. Openbox y Chromium

Openbox es un gestor de ventanas ligero que permite iniciar un entorno gráfico básico sin consumir demasiados recursos. Chromium fue utilizado como navegador para mostrar el dashboard web personalizado en modo kiosco al iniciar el sistema.

3.6. Herramientas Integradas

- **Nmap:** herramienta de exploración y descubrimiento de hosts dentro de una red.
- **Tcpdump:** herramienta para captura y análisis de paquetes de red desde consola.
- **Htop:** monitor interactivo de procesos y recursos del sistema.
- **Iftop:** herramienta para monitorear tráfico de red en tiempo real.
- **Docker:** plataforma para la ejecución de contenedores.
- **Traceroute:** herramienta para analizar la ruta que siguen los paquetes hacia un destino.
- **Net-tools:** conjunto de utilidades clásicas de red como ifconfig y netstat.

4. Desarrollo del Proyecto

4.1. Creación de la Máquina Virtual de Desarrollo

El desarrollo inició con la creación y parametrización de una máquina virtual en Virtual-Box que sirvió como entorno de compilación para la ISO personalizada.

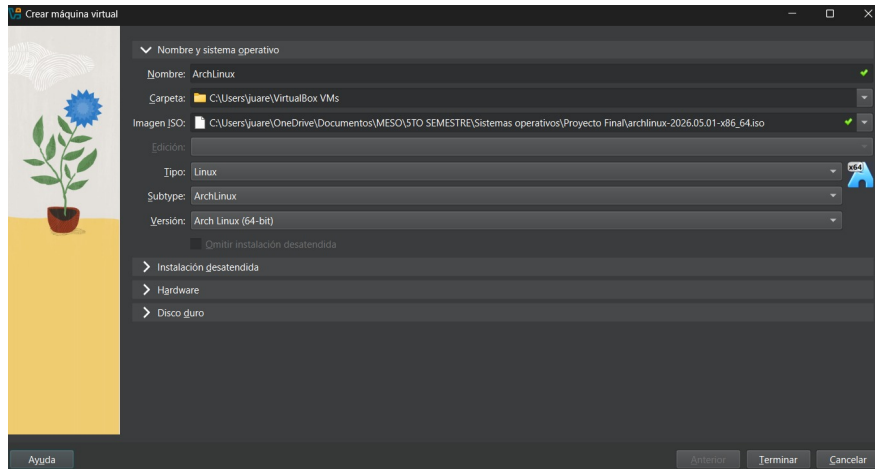


Figura 1: Selección de la ISO base de Arch Linux y tipo de OS en VirtualBox.

Para garantizar un proceso de compilación fluido y emular configuraciones modernas, se asignaron 8 GB de memoria RAM, 4 núcleos de CPU y soporte para arranque EFI.

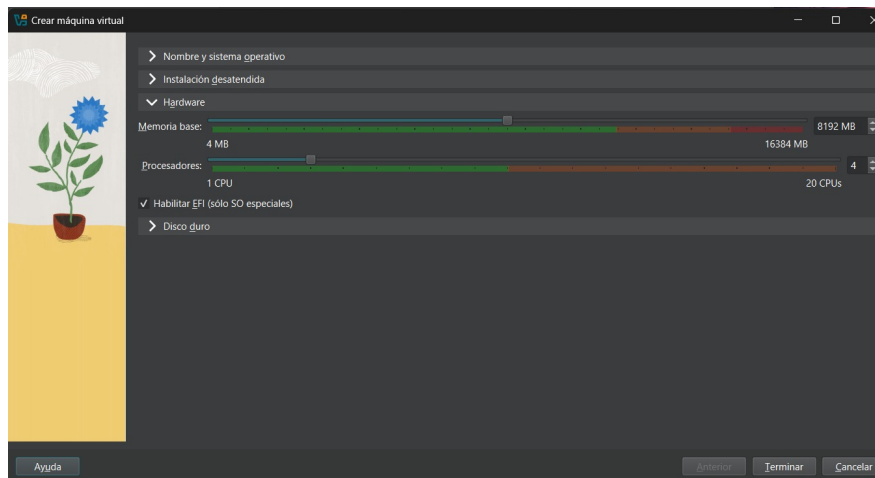


Figura 2: Asignación de hardware virtualizado (RAM, CPU y EFI habilitado).

El almacenamiento se configuró mediante un disco duro virtual de expansión dinámica con un tamaño máximo de 40 GB para alojar el sistema base, las herramientas y los archivos temporales de empaquetado.

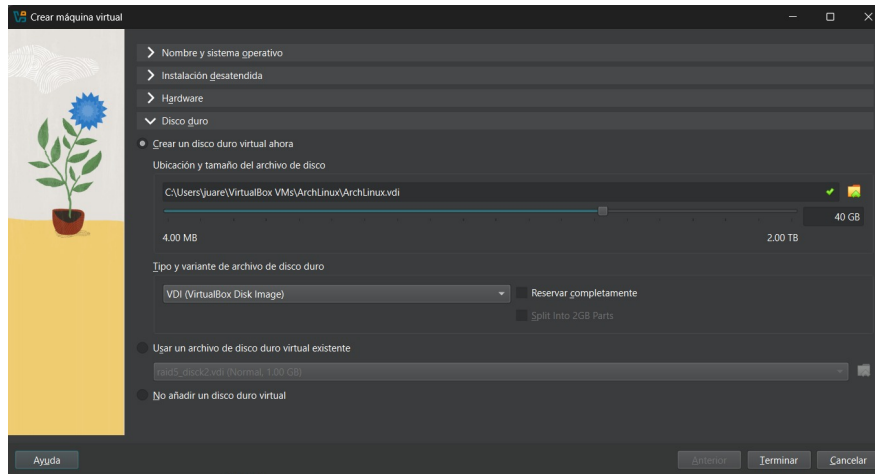


Figura 3: Configuración del disco duro virtual para el entorno de desarrollo.

4.2. Instalación del Sistema Base de Arch Linux

La instalación del sistema base en el entorno de desarrollo se facilitó a través del script guiado oficial `archinstall`, seleccionando un perfil minimalista para evitar paquetería huérfana u opcional.

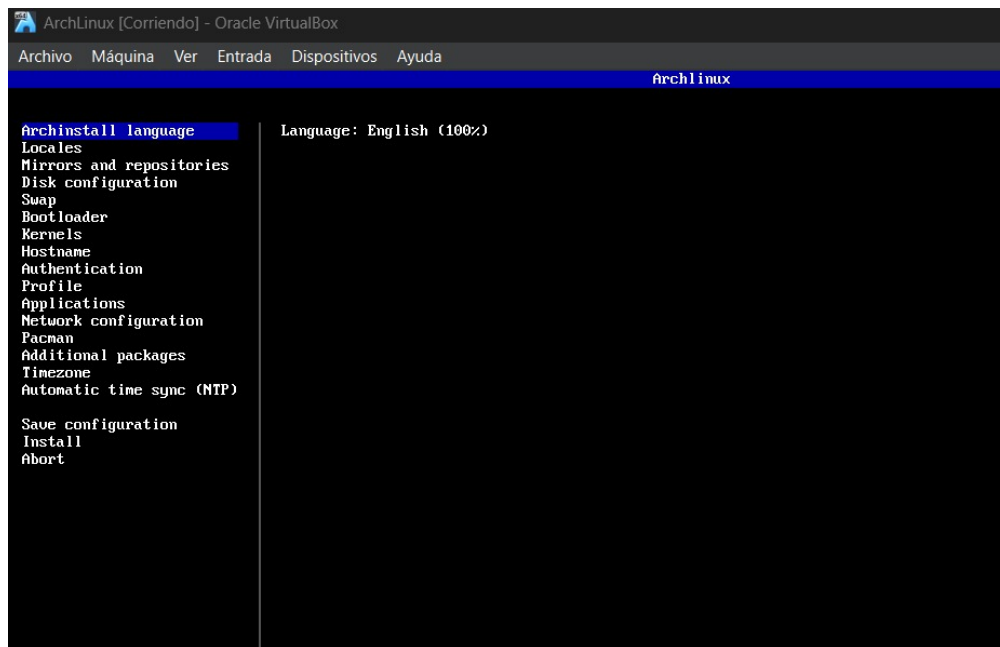


Figura 4: Menú interactivo de configuración en la utilidad `archinstall`.

El script completó la instalación del sistema base de manera limpia en un tiempo óptimo,

dejando el disco virtual listo para el primer inicio.

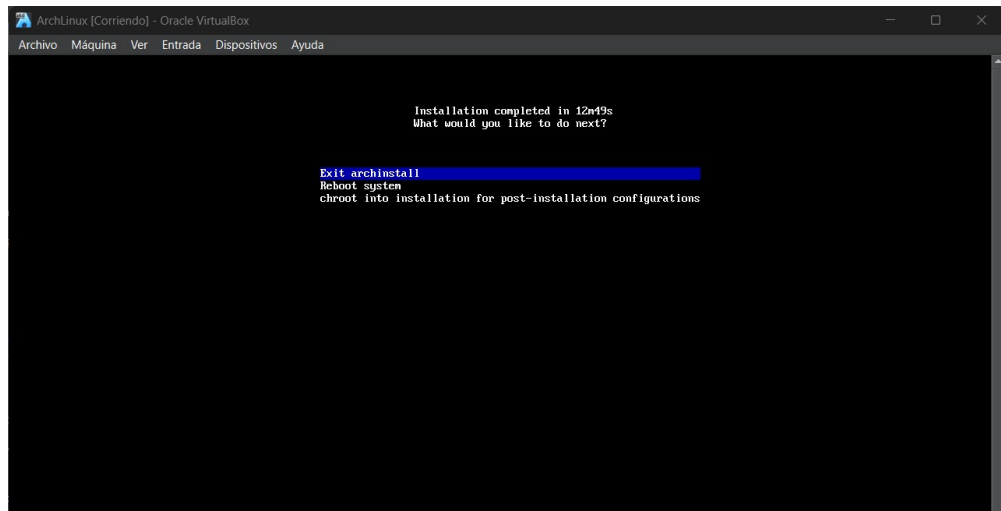


Figura 5: Culminación exitosa del proceso de instalación base.

4.3. Configuración de Usuario y Sincronización Inicial

Tras retirar el medio de instalación virtual, se arrancó el sistema desde el disco duro local, iniciando sesión en la consola `tty1` con las credenciales del entorno local bajo el hostname `gioyosh-so`.

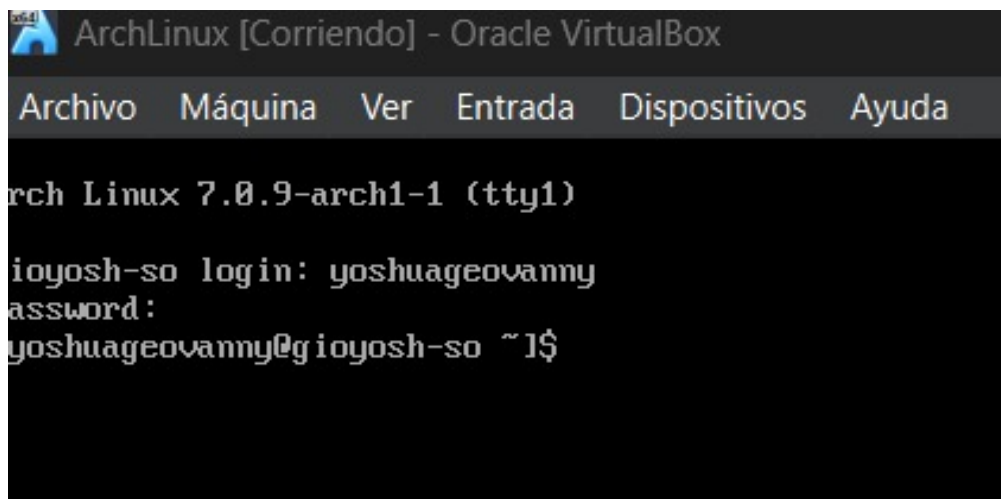


Figura 6: Primer arranque del sistema base e inicio de sesión por terminal.

Como paso fundamental previo al despliegue de ArchISO, se ejecutó una sincronización de los repositorios oficiales mediante el gestor de paquetes de la distribución.

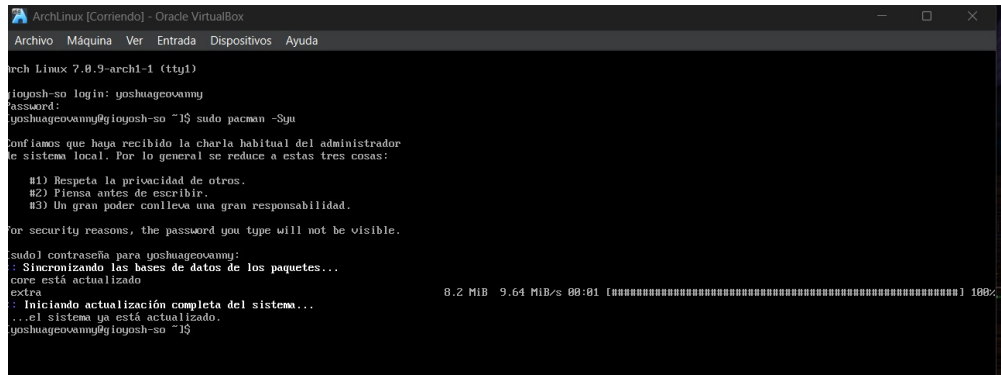


Figura 7: Sincronización y actualización global mediante `sudo pacman -Syu`.

4.4. Instalación de Utilidades del Sistema y Entorno ArchISO

Se instalaron herramientas generales de edición de texto y clonación de repositorios necesarios para manipular la estructura interna del Live ISO.

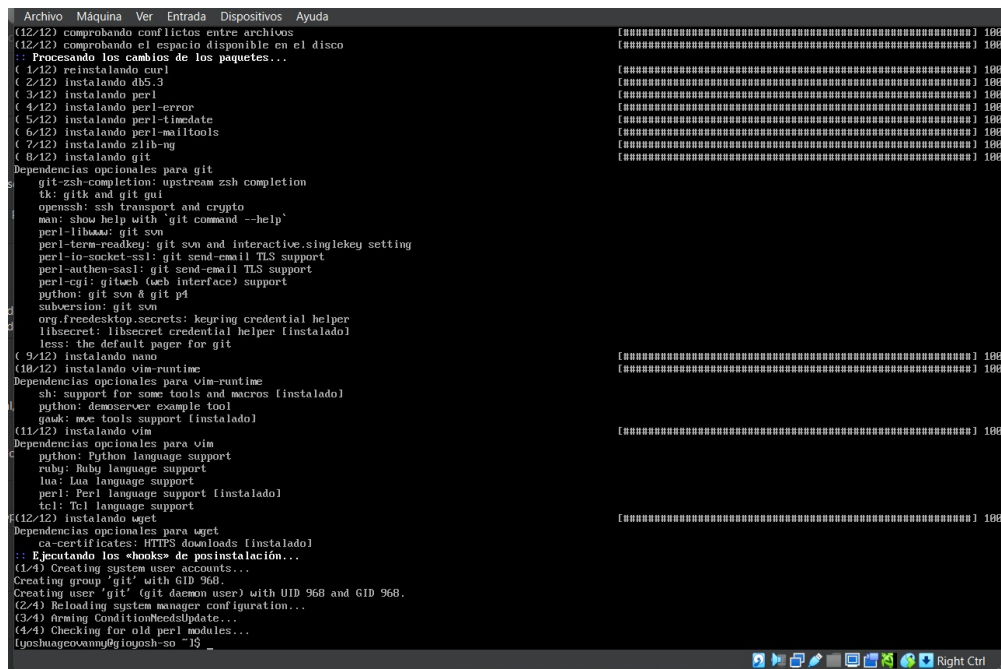


Figura 8: Instalación de paquetes utilitarios indispensables (`git`, `nano`, `vim`, `wget`).

Posteriormente, se instaló la suite de empaquetado de distribuciones `archiso`, la cual proporciona la infraestructura de scripts base para generar imágenes autoejecutables.

```
lyoshuagovanny@iogosh-so ~]$ pacman -Qi archiso
Nombre      : archiso
Versión     : 88-1
Descripción : Tools for creating Arch Linux live and install iso images
Arquitectura : any
URL         : https://gitlab.archlinux.org/archlinux/archiso
Licencias   : GPL-3.0-or-later
Grupos      : Nada
Provee      : Nada
Depende de  : arch-install-scripts bash dosfstools c2fsprogs erofs-utils libarchive libisoburn mtools squashfs-tools
Dependencias opcionales :
  cdK2-ovmf: for emulating UEFI with run_archiso
  gnupg: for OpenPGP signature verification of rootfs over PXE [instalado]
  grub: for grub support in the ISO [instalado]
  openssl: for CMS signature verification of PXE artifacts and rootfs over PXE [instalado]
  qemu-desktop: for run_archiso
Exigido por  : Nada
Opcional para : Nada
En conflicto con : Nada
Reemplaza a  : Nada
Tamaño de la instalación : 225.49 KiB
Encargado    : Daniel Bermond <dbermond@archlinux.org>
Fecha de creación : vie 27 mar 2025 08:36:01
Fecha de instalación : vie 22 may 2025 10:21:47
Motivo de la instalación : Instalado explícitamente
Guión de instalación : No
Validado por  : Firma
```

Figura 9: Verificación de la instalación explícita del paquete archiso.

4.5. Estructuración del Perfil Live y Selección de Paquetes

El perfil base `releng` fue copiado al espacio de trabajo. Se procedió a editar la lista atómica de paquetes en el archivo de configuración `packages.x86_64` mediante el editor `nano`, declarando únicamente las dependencias críticas de red y entorno gráfico.

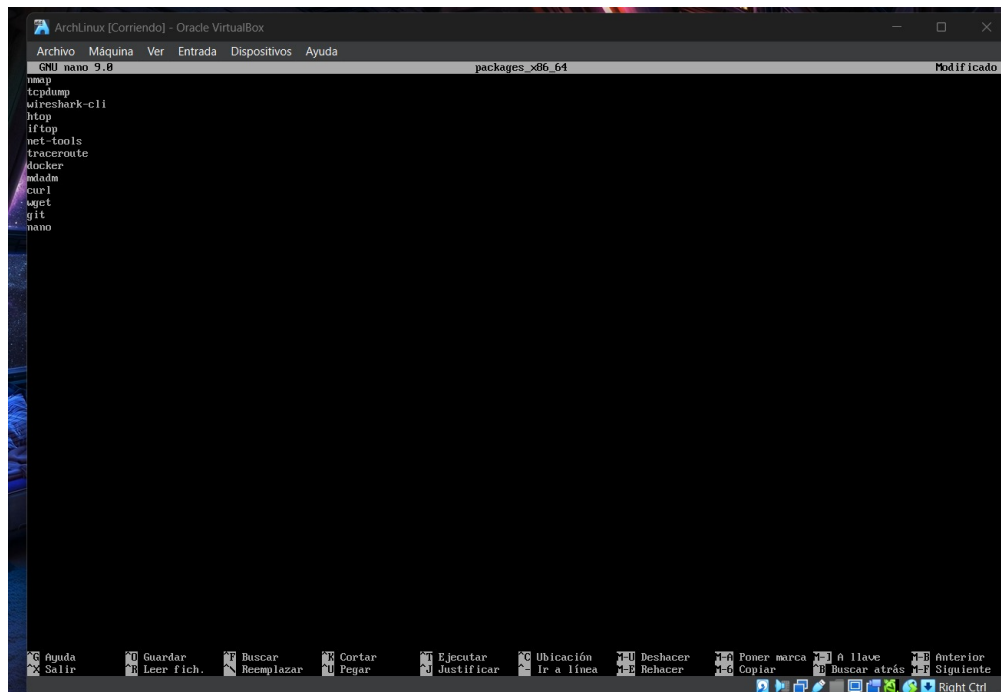


Figura 10: Definición de las herramientas de red y entorno gráfico en `packages.x86_64`.

Para consolidar la lógica del sistema operativo, se estableció una correlación exacta entre los módulos operativos planteados en el diseño y los binarios empaquetados en la distribución:

Función	Herramienta
Escaneo	nmap
Captura	tcpdump
Análisis	tshark (wireshark-cli)
Monitoreo	htop, iftop
Diagnóstico	traceroute, net-tools
Contenedores	docker
RAID	mdadm

Figura 11: Matriz de componentes de software de la distribución.

4.6. Automatización con Systemd y Scripts de Inicialización

Para evitar la saturación del almacenamiento del sistema de archivos en RAM (*OverlayFS*) debido a capturas continuas de tráfico, se diseñó un servicio de limpieza cronometrado.

```

ArchLinux [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
GNU nano 9.8                                profiledef.sh
#!/usr/bin/env bash
# shellcheck disable=SC2034

iso_name="netguardian"
iso_label="NETGUARDIAN_${date --date="@${SOURCE_DATE_EPOCH}-${date +%s}" +%Y%m}"
iso_publisher="GioYosh SD Project"
iso_application="NetGuardian OS - Network Monitoring and Diagnostics"
iso_version="${date --date="@${SOURCE_DATE_EPOCH}-${date +%s}" +%Y.%m.%d}"
install_dir="arch"
buildmodes=('iso')
bootmodes=('bios.syslinux'
            'uefi.systemd-boot')
pacman_conf="pacman.conf"
airootfs_image_type="squashfs"
airootfs_image_tool_options=('-comp' 'xz' '-Xbcj' 'x86' '-b' '1M' '-Xdict-size' '1M')
bootstrap_tarball_compression=('zstd' '-c' '-T0' '--auto-threads=logical' '--long' '-19')
file_permissions=(
  ["/etc/shadow"]="0:0:400"
  ["/root"]="0:0:750"
  ["/root/.automated_script.sh"]="0:0:755"
  ["/root/.gnupg"]="0:0:700"
  ["/usr/local/bin/choose-mirror"]="0:0:755"
  ["/usr/local/bin/installation_guide"]="0:0:755"
  ["/usr/local/bin/livecd-sound"]="0:0:755"
)

```

Figura 12: Archivo de unidad de systemd para el servicio de limpieza automática.

Este servicio se emparejó con un temporizador activo (**timer**) para asegurar ejecuciones

recurrentes de mantenimiento de almacenamiento cada 15 minutos.

```
groups (981)
tty (5)
dbus (81)
utmp (996)
games (58)
git (968)
uid (971)
unknown (34)
unknown (32)
systemd-network (977)
tss (972)
systemd-journal-remote (975)
systemd-journal (988)
[tmkarchiso] INFO: Creating checksum file for self-test...
[tmkarchiso] INFO: Done!
[tmkarchiso] INFO: Creating ISO image...
xorriso 1.5.8 : RockRidge filesystem manipulator, libburnia project.

xorriso : NOTE : Environment variable SOURCE_DATE_EPOCH encountered with value 1779469719
Drive current: -outdev 'stdio:/home/yoshuageovanny/NetGuardianOS/out/netguardian-2026.05.22-x86_64.iso'
Media current: stdio file, overwriteable
Media status : is blank
Media summary: 0 sessions, 0 data blocks, 0 data, 38.2g free
xorriso : WARNING : -valid text is too long for Joliet (18 > 16)
Added to ISO image: directory '/' = '/home/yoshuageovanny/NetGuardianOS/work/iso'
xorriso : UPDATE : 187 files added in 1 seconds
xorriso : UPDATE : 187 files added in 1 seconds
xorriso : NOTE : Copying to System Area: 432 bytes from file '/home/yoshuageovanny/NetGuardianOS/work/iso/boot/syslinux/isohdpxx.bin'
libisofs: NOTE : Automatically adjusted MBR geometry to 1821/77/32
libisofs: NOTE : Aligned image size to cylinder size by 227 blocks
xorriso : UPDATE : 15.68% done
xorriso : UPDATE : 86.41% done
ISO image produced: 768536 sectors
Written to medium: 768536 sectors at LBA 0
Writing to 'stdio:/home/yoshuageovanny/NetGuardianOS/out/netguardian-2026.05.22-x86_64.iso' completed successfully.

[tmkarchiso] INFO: Done!
1.56 /home/yoshuageovanny/NetGuardianOS/out/netguardian-2026.05.22-x86_64.iso
[yoshuageovanny@gioyosh-so NetGuardianOS]$ ls out/
netguardian-2026.05.22-x86_64.iso
[yoshuageovanny@gioyosh-so NetGuardianOS]$
```

Figura 13: Temporizador systemd configurado para la purga de trazas.

Adicionalmente, se inyectaron scripts de automatización en la ruta `airootfs/root/init-netmonitor.sh` encargados de habilitar el forwarding IP de la pila de red del Kernel y levantar los contenedores de monitoreo al vuelo.

```
xdg-utils
xfsprogs
x12tpd
zsh
netdata

[yoshuageovanny@gioyosh-so NetGuardianOS]$
```

Figura 14: Script Bash embebido para la configuración reactiva de red en el arranque.

Para orquestar el entorno gráfico liviano, se configuró el servicio encargado de invocar automáticamente el servidor Xorg y cargar el entorno gráfico Kiosco.

4.8. Compilación y Generación de NetGuardian OS

Con la estructura de archivos (`airootfs`), los scripts de automatización y las configuraciones de servicios en su lugar dentro del perfil del proyecto, se invocó la utilidad de compilación `mkarchiso` en modo detallado.

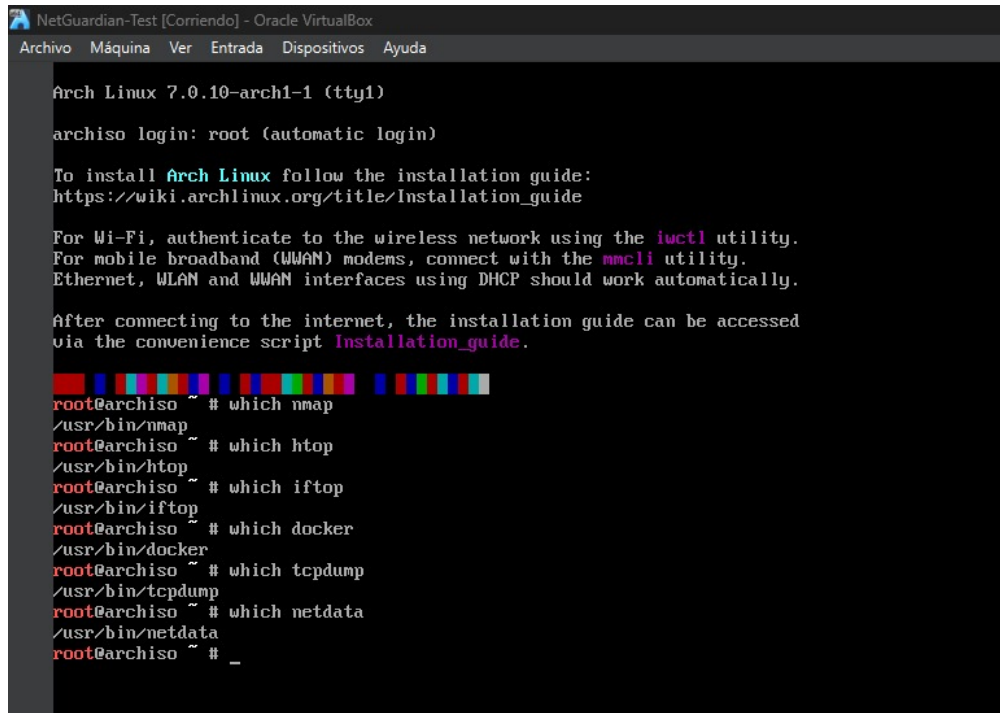


Figura 17: Lanzamiento de la compilación de la imagen ISO personalizada.

El árbol final del perfil de construcción refleja la estructura organizada del proyecto, listando el directorio raíz modificado que se grabará directamente en la imagen final.

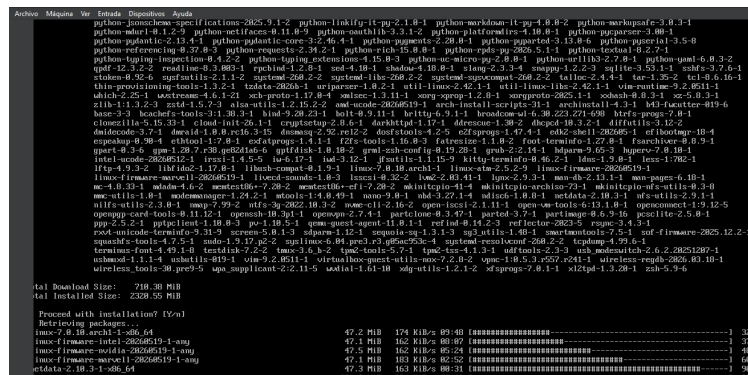


Figura 18: Árbol de directorios de personalización del perfil de empaquetado.

5. Pruebas Realizadas y Diagnóstico

5.1. Validación de Interfaces de Red y Contenedores

Al bootear la imagen ISO generada, se procedió a verificar el mapa de direccionamiento IP local para corroborar que las subredes de docker no colisionaran con las interfaces físicas bridge asignadas por la máquina virtual.

```

root@netguardian ~# docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
root@netguardian ~# systemctl status docker
* docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; disabled; preset: disabled)
   Active: active (running) since Sun 2026-05-31 22:13:46 UTC; 5min ago
   Invocation: 89ca32fe3073473f945f40df798355f5
   TriggeredBy: * docker.socket
   Docs: https://docs.docker.com
   Main PID: 1091 (dockerd)
   Tasks: 10
   Memory: 151.6M (peak: 165.9M)
   CPU: 2.914s
   CGroup: /system.slice/docker.service
           └─1091 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

May 31 22:13:46 netguardian dockerd[1091]: time="2026-05-31T22:13:46.059004697Z" level=warning msg="git source cannot be enabled: failed to find git binary: ex
May 31 22:13:46 netguardian dockerd[1091]: time="2026-05-31T22:13:46.059954562Z" level=info msg="Completed buildkit initialization"
May 31 22:13:46 netguardian dockerd[1091]: time="2026-05-31T22:13:46.065769962Z" level=info msg="Dacemon has completed initialization"
May 31 22:13:46 netguardian dockerd[1091]: time="2026-05-31T22:13:46.065956062Z" level=info msg="API listen on /run/docker.sock"
May 31 22:13:46 netguardian systemd[1]: Started Docker Application Container Engine.
May 31 22:14:46 netguardian dockerd[1091]: time="2026-05-31T22:14:46.396462201Z" level=info msg="image pulled" digest="sha256:5aca99593157f4ae539a5dc1092a0a08
May 31 22:14:46 netguardian dockerd[1091]: time="2026-05-31T22:14:46.449524966Z" level=error msg="Error saving dying container to disk: invalid output path: st
May 31 22:14:46 netguardian dockerd[1091]: time="2026-05-31T22:14:46.449684705Z" level=error msg="Handler for POST /containers/create returned error" error=res
May 31 22:16:58 netguardian dockerd[1091]: time="2026-05-31T22:16:58.520189461Z" level=error msg="Error saving dying container to disk: invalid output path: st
May 31 22:16:58 netguardian dockerd[1091]: time="2026-05-31T22:16:58.520461194Z" level=error msg="Handler for POST /containers/create returned error" error=res
tues 1-23/23 (END)
```

Figura 19: Verificación de direccionamiento de interfaces nativas y virtuales.

Al comprobar las interfaces, se validó el levantamiento correcto de las imágenes de contenedores descargadas durante la inicialización en vivo.

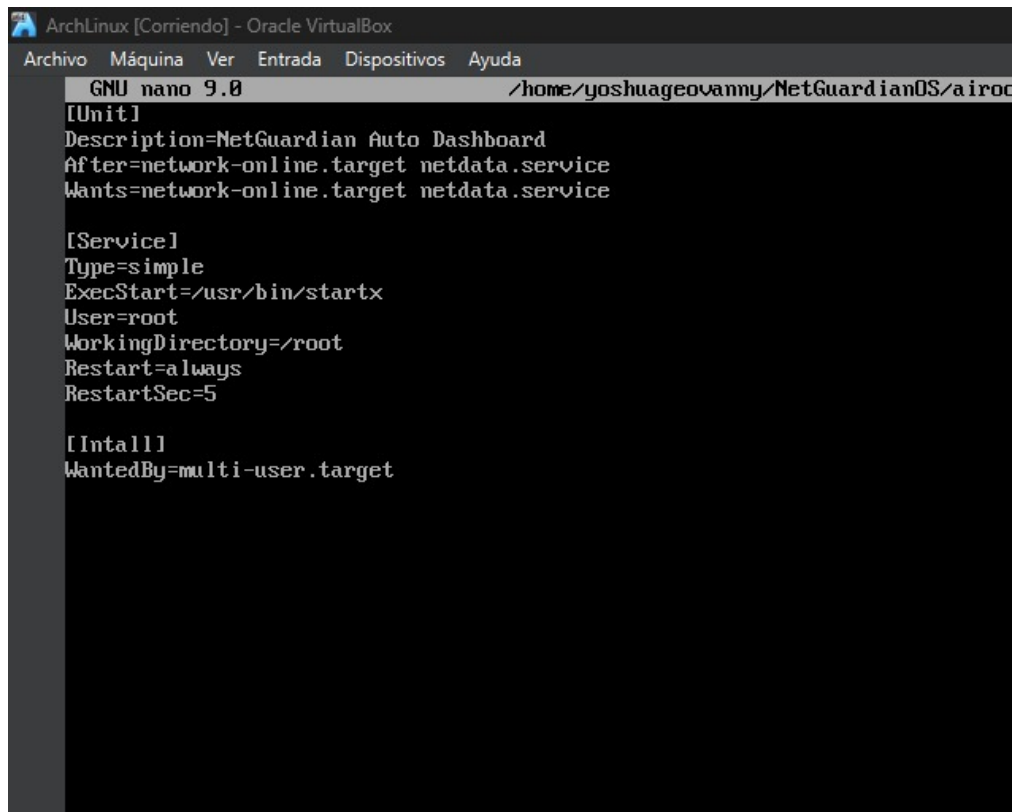
```

[yoshuageovanny@gioyosh-so NetGuardianOS1$ cat ~/NetGuardianOS/airootfs/etc/hostname
archiso
[yoshuageovanny@gioyosh-so NetGuardianOS1$ echo netguardian > ~/NetGuardianOS/airootfs/etc/hostname
[yoshuageovanny@gioyosh-so NetGuardianOS1$ cat ~/NetGuardianOS/airootfs/etc/hostname
netguardian
[yoshuageovanny@gioyosh-so NetGuardianOS1$
```

Figura 20: Ejecución e inicialización de contenedores mediante Docker Compose.

5.2. Captura Atómica y Análisis en Vivo

Se ejecutaron pruebas directas con el binario `tcpdump` para evaluar la sensibilidad del sniffer embebido, detectando de forma transparente tráfico de multidifusión local (MDNS), peticiones ARP y resoluciones de dominio DNS.



```
ArchLinux [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
GNU nano 9.0 /home/yoshuageovanny/NetGuardianOS/airoc
[Unit]
Description=NetGuardian Auto Dashboard
After=network-online.target netdata.service
Wants=network-online.target netdata.service

[Service]
Type=simple
ExecStart=/usr/bin/startx
User=root
WorkingDirectory=/root
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Figura 21: Captura activa de tráfico de red en la interfaz nativa de diagnóstico.

Para validar el aislamiento e interconexión funcional de las tres capas core del sistema (Sistema base, Capa de red y Servicios de aplicación), se esquematizó el flujo de conectividad interna del proyecto:

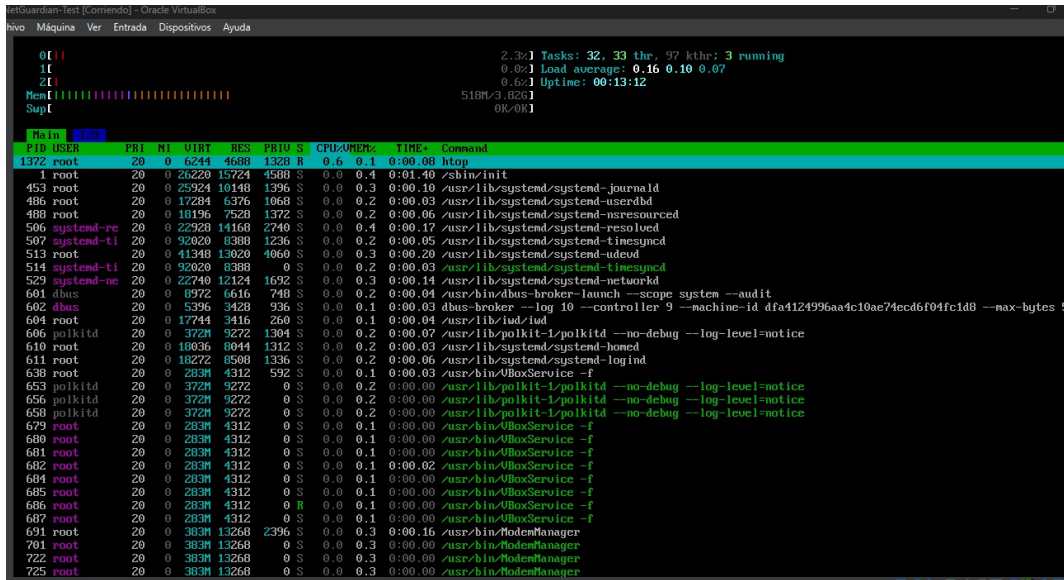


Figura 22: Arquitectura en capas y flujo de comunicación de NetGuardian OS.

5.3. Interfaz de Usuario Final (Dashboard)

El resultado definitivo de la personalización gráfica se evidencia al cargar el navegador Chromium en modo kiosco sin bordes, desplegando un centro de control unificado optimizado para la pantalla de diagnóstico del administrador de red.



Figura 23: Dashboard web de control de NetGuardian OS.

6. Problemas Encontrados y Soluciones

6.1. Problemas con Mirrors durante la Instalación

Durante la instalación de Arch Linux se presentaron errores de descarga debido a servidores espejo lentos. El mensaje indicaba fallos al recuperar paquetes desde el repositorio. La solución consistió en cambiar la selección de mirrors y repetir el proceso de instalación hasta finalizar correctamente.

6.2. Arranque desde la ISO de Instalación

Después de instalar Arch Linux, la máquina virtual continuaba arrancando desde la ISO original de instalación. Esto ocurrió porque la imagen ISO seguía montada en VirtualBox. La solución fue retirar la ISO desde la configuración de almacenamiento de la máquina virtual, permitiendo que el sistema arrancara desde el disco virtual instalado.

6.3. Problemas con la Carpeta Compartida

Al intentar copiar la ISO generada hacia Windows, se presentaron errores de permisos en la carpeta compartida de VirtualBox. La solución fue utilizar permisos de administrador mediante `sudo` y verificar el montaje correcto de la carpeta compartida en `/media/sf.ISOS`.

6.4. Uso Inicial de Netdata

Inicialmente se integró Netdata como herramienta de dashboard. Sin embargo, se determinó que Netdata proporcionaba automáticamente gran parte de las funcionalidades requeridas, incluyendo recolección de métricas, visualización y monitoreo en tiempo real.

Debido a que el proyecto no permitía utilizar una herramienta que resolviera completamente el propósito principal, se decidió retirar Netdata de la versión final y desarrollar un dashboard propio para NetGuardian OS. Esta decisión permitió cumplir mejor con los lineamientos académicos, ya que el dashboard final fue diseñado por los estudiantes e integrado directamente dentro de la distribución personalizada.

6.5. Problemas con Docker en Entorno Live

Durante las pruebas se verificó que Docker estaba instalado correctamente; sin embargo, al intentar ejecutar contenedores se presentaron errores relacionados con OverlayFS. Esto se debe a que la ISO Live utiliza un sistema de archivos temporal y no persistente. Docker fue conservado como herramienta disponible dentro de la distribución, pero se documentó que su uso completo requiere almacenamiento persistente o instalación en disco.

7. Resultados Obtenidos

Como resultado del proyecto se logró desarrollar una distribución Linux personalizada funcional, capaz de arrancar desde una imagen ISO e integrar diversas herramientas para monitoreo y diagnóstico de redes.

NetGuardian OS permite realizar tareas de exploración de red, captura de tráfico, monitoreo de procesos, análisis de consumo de recursos y visualización inicial mediante un dashboard

propio. El sistema arranca automáticamente, inicia un entorno gráfico ligero y muestra una interfaz web personalizada, facilitando su uso en entornos de diagnóstico rápido.

8. Conclusiones

1. Se logró diseñar e implementar exitosamente una distribución Linux personalizada basada en Arch Linux para tareas de monitoreo y diagnóstico de redes.
2. La utilización de ArchISO permitió generar una imagen ISO funcional y adaptable a los requerimientos del proyecto.
3. La integración de herramientas especializadas permitió realizar análisis de red, captura de tráfico y monitoreo de recursos del sistema.
4. El uso de systemd facilitó la automatización del arranque y la ejecución de servicios dentro de la distribución.
5. La eliminación de Netdata como herramienta principal permitió desarrollar una solución más propia, cumpliendo con la restricción de no utilizar aplicaciones que resolvieran completamente el proyecto.
6. El dashboard personalizado permitió presentar una interfaz inicial sencilla y comprensible para el usuario final.

9. Recomendaciones

- Implementar almacenamiento persistente para mejorar el funcionamiento de Docker dentro de la distribución.
- Agregar scripts adicionales para automatizar escaneos de red mediante Nmap.
- Mejorar el dashboard para mostrar métricas dinámicas en tiempo real.
- Probar la ISO en hardware físico mediante una memoria USB booteable.
- Documentar cada fase del proyecto con capturas y evidencias técnicas.

10. Bibliografía

- Arch Linux. *Arch Linux Documentation*.
- Arch Linux Wiki. *ArchISO*.
- Arch Linux Wiki. *Systemd*.
- Oracle. *VirtualBox User Manual*.
- Nmap. *Nmap Network Scanning Documentation*.
- Tcpdump. *Tcpdump Official Documentation*.
- Docker. *Docker Documentation*.
- GNU/Linux Manual Pages.